ANALYSIS OF ALGORITHMS

Comprehensive Guide on Time Complexity & Recurrence Relations

Q1. Determine the time complexity:

```
for(int i=0; i<n; i++) {
    cout<<i;
}
```

Time Complexity: ***O(n)***

Explanation: The loop initializes at ***i = 0*** and increments by 1 in each iteration until it reaches ***n***. The statement inside executes exactly ***n*** times, making the time complexity linear.

Q2. Determine the time complexity:

```
for(int i=0; i<n; i++) {
    for(int j=0; j<n; j++) {
        cout<<"*";
    }
}
```

Time Complexity: ***O(n²)***

Explanation: This consists of nested loops. The outer loop runs ***n*** times, and for each iteration of the outer loop, the inner loop also runs ***n*** times. Total iterations = ***n × n = n²***.

Q3. Determine the time complexity:

```
for(int i=0; i<n; i++) {
    for(int j=0; j<i; j++) {
        cout<<"*";
    }
}
```

Time Complexity: ***O(n²)***

Explanation: The inner loop runs ***i*** times where ***i*** ranges from 0 to ***n-1***. Total work done is the sum of the first ***n-1*** integers: ***0 + 1 + 2 + ... + (n-1) = [n(n-1)] / 2***, which asymptotically simplifies to ***O(n²)***.

Q4. Determine the time complexity:

```
for(int i=1; i<n; i*=2) {
    cout<<i;
}
```

Time Complexity: ***O(log n)***

Explanation: The loop variable ***i*** doubles each time (***1, 2, 4, 8, ..., 2^k***). The loop stops when ***$2^k \geq n$***, which implies ***$k = log_2(n)$*** iterations.

Q5. Determine the time complexity:

```
for(int i=n; i>1; i/=2) {
    cout<<i;
}
```

Time Complexity: ***O(log n)***

Explanation: The loop starts at ***n*** and divides ***i*** by 2 continuously until ***$i \leq 1$***. This is the inverse of doubling and takes exactly ***$log_2(n)$*** steps.

Q6. Determine the time complexity:

```
for(int i=1; i<=n; i++) {
    for(int j=1; j<=log(n); j++) {
        cout<<"*";
    }
}
```

Time Complexity: ***O(n log n)***

Explanation: The outer loop runs ***n*** times. The inner loop executes independent of ***i*** and runs exactly ***log(n)*** times for every outer loop iteration. Multiplying them gives ***O(n log n)***.

Q7. Determine the time complexity:

```
while (n > 0) {
    n--;
}
```

Time Complexity: ***O(n)***

Explanation: The loop decrements ***n*** by 1 in each step until ***n*** drops to 0. It executes exactly ***n*** times.

Q8. Determine the time complexity:

```
while (n > 1) {
    n /= 2;
}
```

Time Complexity: ***O(log n)***

Explanation: In each iteration, the value of ***n*** is halved. The total number of steps to reduce ***n*** to 1 is $log_2(n)$.

Q9. Determine the time complexity:

```
for(int i=1; i<=n; i++) {}
for(int j=1; j<=n; j++) {}
```

Time Complexity: ***O(n)***

Explanation: These loops are sequential, not nested. The first loop takes ***O(n)*** time and the second loop takes ***O(n)*** time. Total time complexity is $O(n) + O(n) = O(n)$.

Q10. Determine the time complexity:

```
for(int i=1; i<=n; i++) {}
for(int j=1; j<=n*n; j++) {}
```

Time Complexity: $O(n^2)$

Explanation: These loops are sequential. The first takes ***O(n)*** steps and the second takes $O(n^2)$ steps. In asymptotic analysis, the higher-order term dominates: $O(n + n^2) = O(n^2)$.

Q11. Determine the time complexity:

```
for(int i=1; i<=n; i++) {
    for(int j=1; j<=n*n; j++) {}
}
```

Time Complexity: $O(n^3)$

Explanation: These are nested loops. The outer loop executes ***n*** times. For every iteration, the inner loop executes n^2 times. Total operations = $n \times n^2 = n^3$.

Q12. Determine the time complexity:

```
for(int i=1; i<=n; i*=3) {}
```

Time Complexity: $O(\log_3 n)$ or simply $O(\log n)$

Explanation: The loop control variable i is multiplied by 3 each time ($1, 3, 9, 27, ...$). It takes $\log_3(n)$ iterations to reach or exceed n. Base change formula preserves $O(\log n)$.

Q13. Determine the time complexity:

```
for(int i=1; i<=n; i++) {
    for(int j=1; j<=i*i; j++) {}
}
```

Time Complexity: $O(n^3)$

Explanation: The inner loop executes i^2 times for each i. Summing this from 1 to n gives: $\Sigma\, i^2 = [n(n+1)(2n+1)] / 6$, which is a third-degree polynomial, giving an overall asymptotic complexity of $O(n^3)$.

Q14. Determine the time complexity:

```
void fun(int n) {
    if(n==0) return;
    fun(n-1);
}
```

Time Complexity: $O(n)$

Explanation: The function reduces n by 1 recursively until hitting the base case $n=0$. The recurrence relation is $T(n) = T(n-1) + O(1)$, which evaluates to $O(n)$.

Q15. Determine the time complexity:

```
void fun(int n) {
    if(n<=1) return;
    fun(n/2);
}
```

Time Complexity: $O(\log n)$

Explanation: The input n is halved during each recursive call. The recurrence relation is $T(n) = T(n/2) + O(1)$. By Master's Theorem or substitution, this yields $O(\log n)$.

Q16. Determine the time complexity:

```
void fun(int n) {
    if(n==0) return;
    fun(n-1);
    fun(n-1);
}
```

Time Complexity: $O(2^n)$

Explanation: Each function call spawns 2 additional recursive calls, decreasing n by 1. The recurrence is $T(n) = 2T(n-1) + O(1)$. The recursion tree reaches a depth of n, creating a total of $2^n - 1$ nodes, hence exponential time.

Q17. Determine the time complexity:

```
int fib(int n) {
    if(n<=1) return n;
    return fib(n-1) + fib(n-2);
}
```

Time Complexity: $O(2^n)$ (More tightly, $O(1.618^n)$)

Explanation: This represents the naive Fibonacci sequence implementation. The recurrence is $T(n) = T(n-1) + T(n-2) + O(1)$. The growth tracks the golden ratio ($\varphi \approx 1.618$), bounded asymptotically by $O(2^n)$.

Q18. Determine the time complexity:

```
int fact(int n) {
    if(n<=1) return 1;
    return n * fact(n-1);
}
```

Time Complexity: $O(n)$

Explanation: This performs a single linear chain of n recursive calls to calculate the factorial. Recurrence: $T(n) = T(n-1) + O(1)$, which simplifies directly to linear time $O(n)$.

Q19. Determine the time complexity:

```
void fun(int n) {
    if(n<=0) return;
    fun(n-1);
    cout<<n;
}
```

Time Complexity: ***O(n)***

Explanation: Printing a single value takes ***O(1)*** time. The execution path forms a linear recursion identical to Q14 and Q18, calling the function exactly ***n+1*** times. Thus, complexity remains ***O(n)***.

Q20. Determine the time complexity:

```
void fun(int n) {
    if(n<=1) return;
    fun(n/2);
    cout<<n;
}
```

Time Complexity: ***O(log n)***

Explanation: Just like Q15, the work done per level is ***O(1)*** (printing a value), and the problem size is halved at each step. This yields $log_2(n)$ calls, hence ***O(log n)***.

Section 2: Solving Recurrence Relations

Q21. Solve the recurrence relation: ***T(n) = T(n-1) + 1*** **with base case** ***T(0) = 1***

Solution: ***T(n) = O(n)***

Derivation (Substitution Method):
T(n) = T(n-1) + 1
Substitute ***T(n-1) = T(n-2) + 1*** → ***T(n) = [T(n-2) + 1] + 1 = T(n-2) + 2***
Generalizing after ***k*** steps: ***T(n) = T(n-k) + k***
To reach the base case, let ***n - k = 0*** → ***k = n***.
Therefore, ***T(n) = T(0) + n = 1 + n***, which matches ***O(n)***.

Q22. Solve the recurrence relation: $T(n) = T(n-1) + n$

Solution: $T(n) = O(n^2)$

Derivation:

Expanding the equation step-by-step:

$T(n) = T(n-1) + n$

$T(n) = [T(n-2) + (n-1)] + n = T(n-2) + n + (n-1)$

Continuing until the base case: $T(0) + 1 + 2 + ... + n$

This is the sum of the first n natural numbers: $[n(n+1)] / 2$, which yields $O(n^2)$.

Q23. Solve the recurrence relation: $T(n) = T(n/2) + 1$

Solution: $T(n) = O(\log n)$

Derivation:

By Master's Theorem for dividing recurrences: $T(n) = aT(n/b) + f(n)$, where $a = 1, b = 2, f(n) = 1$.

Compute $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.

Since $f(n) = \Theta(1)$, this falls into **Case 2** of the Master Theorem.

Thus, $T(n) = \Theta(1 \times \log n) = O(\log n)$.

Q24. Solve the recurrence relation: $T(n) = 2T(n-1) + 1$

Solution: $T(n) = O(2^n)$

Derivation:

Expanding by substitution:

$T(n) = 2[2T(n-2) + 1] + 1 = 4T(n-2) + 2 + 1$

$T(n) = 8T(n-3) + 4 + 2 + 1$

After k steps: $T(n) = 2^k T(n-k) + (2^{k-1} + ... + 4 + 2 + 1)$

Setting $k = n$ gives a geometric series sum: $2^n T(0) + (2^n - 1)$, which is $O(2^n)$.

Q25. Solve the recurrence relation: $T(n) = T(n/2) + n$

Solution: $T(n) = O(n)$

Derivation:

Using Master's Theorem: $a = 1, b = 2, f(n) = n$.

Compute $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.

Compare $f(n) = n^1$ with 1. Since $f(n) = \Omega(n^{0+\varepsilon})$ for $\varepsilon=1$, this falls into **Case 3**.

Hence, the driving function dominates: $T(n) = \Theta(n)$.

Section 3: Bonus GATE-Level Questions

Q26. Determine the complexity:

```
for(int i=1; i<=n; i++) {
    for(int j=1; j<=i; j*=2) {}
}
```

Time Complexity: $O(n)$

Explanation: The inner loop doubles j until $j > i$, running $\log_2(i)$ times for each i. Total runtime = $\Sigma_{i=1}^{n} \log(i) = \log(1) + \log(2) + ... + \log(n) = \log(n!)$.
By Stirling's approximation, $\log(n!) = \Theta(n \log n)$.

Q27. Determine the complexity:

```
for(int i=1; i<=n; i*=2) {
    for(int j=1; j<=n; j++) {}
}
```

Time Complexity: $O(n \log n)$

Explanation: The outer loop multiplies by 2, running exactly $\log_2(n)$ times. For every step of the outer loop, the inner loop executes completely linearly n times. Multiplying them yields $O(n \log n)$.

Q28. Determine the complexity:

```
for(int i=n; i>=1; i--) {
    for(int j=i; j>=1; j--) {}
}
```

Time Complexity: $O(n^2)$

Explanation: The outer loop runs n times downward. The inner loop executes i times. The performance is bounded by the series $n + (n-1) + (n-2) + ... + 1 = [n(n+1)] / 2$, which translates to $O(n^2)$.

Q29. Determine the complexity:

```
for(int i=1; i<=n; i++) {
    for(int j=1; j<=n; j*=2) {}
}
```

Time Complexity: $O(n \log n)$

Explanation: The outer loop iterates linearly n times. The inner loop executes logarithmically ($\log n$ times) independent of the value of i. Combined, they create an overall complexity of $O(n \log n)$.

Q30. Determine the complexity:

```
void fun(int n) {
    if(n<=1) return;
    fun(n/2);
    fun(n/2);
}
```

Time Complexity: $O(n)$

Explanation: The recurrence relation representing this function is $T(n) = 2T(n/2) + O(1)$. Using Master's Theorem: $a = 2, b = 2, f(n) = 1$. $n^{\log_2 2} = n^1 = n$. Since $f(n) = O(n^{1-\varepsilon})$ for $\varepsilon=1$ (Case 1), the complexity is dictated by the leaf nodes: $\Theta(n)$.